Azərbaycan Respublikası Elm və Təhsil Nazirliyi

Qarabağ Universiteti

Mühəndislik Fakültəsi

Verilənlərin Strukturu və Alqoritmlər

Mexatronika və robototexnika mühəndisliyi və Kompüter mühəndisliyi ixtisası üzrə bakalavr proqramının tələbələri üçün.

Laboratoriya işi № 2

Tələbənin adı və soyadı: Ədalət Nəzirli

Qrup: KOM25A

Müəllim: Şamil Hümbətov

Tarix: 21.10.2026

2026

Mövzu: Linked List, Stack və Queue Verilənlər Strukturlarının Tətbiqi, Təhlili və Müqayisəsi

Əsas mənbə: *Mark Allen Weiss, Data Structures and Algorithm Analysis in C++ (4th Edition)*

Məqsəd

Bu laboratoriya işinin məqsədi tələbələrin Linked List, Stack və Queue verilənlər strukturlarının nəzəri əsaslarını öyrənməsi, onları sıfırdan C++ / Python dilində implementasiya etməsi, eksperimental sınaqlar aparması və nəticələri elmi əsaslarla təhlil etməsidir.

Öyrənmə Nəticələri

Laboratoriya işini uğurla tamamlayan tələbə:

- Linked List, Stack və Queue strukturlarının iş prinsiplərini izah edə bilir;
- Verilənlər strukturlarını STL konteynerlərindən istifadə etmədən implementasiya edə bilir;
- Real həyat problemlərini uyğun verilənlər strukturları vasitəsilə modelləşdirə bilir;
- Eksperimental nəticələri cədvəl və qrafiklər vasitəsilə təqdim edə bilir;
- Teorik və praktik mürəkkəblikləri müqayisə edə bilir;
- Performansa təsir edən sistem parametrlərini müəyyən edə bilir.

Tapşırıq 1

Smart Music Playlist (Linked List)

Real Həyat Ssenarisi

Musiqi tətbiqində (Spotify, Apple Music və s.) istifadəçinin playlisti saxlanılır. İstifadəçi mahnı əlavə edə, silə və playlisti görüntüləyə bilir.

A. Araşdırma

Aşağıdakı sualları cavablandırın:

1. Array və Linked List arasında əsas fərqlər hansılardır?

Cavab: Array elementləri yaddaşda ardıcıl yerləşdirir və ölçüsü adətən əvvəlcədən müəyyən olunur; indeksə görə birbaşa giriş O(1)-dir, lakin ortaya əlavə və ya silmə zamanı elementlər sürüşdürülməlidir (O(n)). Linked List-də hər element ayrıca node-dur və növbəti node-a göstərici (pointer/reference) saxlayır; node-lar yaddaşda səpələnmiş ola bilər, ölçü dinamik

dəyişir, mövqe məlum olduqda əlavə/silmə O(1)-dir, amma i-ci elementə çatmaq üçün head-dən başlayıb keçmək lazımdır (O(n)) və hər node əlavə göstərici yaddaşı tələb edir.

2. Niyə musiqi playlistləri üçün Linked List uyğun seçim hesab oluna bilər?

Cavab: Playlist tez-tez dəyişir: mahnılar əlavə olunur, silinir və mahnı sayı əvvəlcədən bilinmir. Linked List-də silmə zamanı digər elementləri sürüşdürmək lazım deyil, yalnız göstəricilər yenidən bağlanır, yaddaş isə yalnız lazım olan qədər ayrılır. Playlist əsasən ardıcıl (bir-birinin ardınca) oxunduğu üçün indeksə görə təsadüfi giriş (random access) vacib deyil və Linked List-in əsas zəifliyi praktikada az hiss olunur.

3. Singly və Doubly Linked List arasında hansı üstünlüklər və çatışmazlıqlar mövcuddur?

Cavab: Singly Linked List-də hər node yalnız next göstəricisi saxlayır: yaddaş az, implementasiya sadədir, lakin yalnız irəli hərəkət mümkündür və əvvəlki elementə qayıtmaq və ya verilmiş node-u silmək üçün əvvəlki node-u tapmaq (O(n)) lazım olur. Doubly Linked List-də next və prev göstəriciləri var: hər iki istiqamətdə gəzmək, "əvvəlki mahnı" funksiyası və node-u O(1)-də silmək mümkündür, amma hər node üçün əlavə yaddaş lazımdır və əlavə/silmə zamanı iki göstərici yenilənməli olduğundan xəta ehtimalı artır.

4. Linked List-də axtarış niyə Array-dən daha yavaş ola bilər?

Cavab: Array-də elementlər ardıcıl yerləşdiyi üçün indeksə görə birbaşa keçid (O(1)) mümkündür, sıralanmışdırsa binary search (O(log n)) tətbiq oluna bilər. Linked List-də isə node-ların ünvanı əvvəlcədən hesablana bilmir, ona görə axtarış həmişə head-dən başlayıb node-ları bir-bir keçməklə (O(n)) aparılır. Bundan əlavə, node-lar yaddaşda səpələnmiş olduğundan prosessor keşindən (cache locality) səmərəli istifadə edilmir.

5. Playlistdə mahnıların sırasını dəyişmək üçün hansı struktur daha əlverişlidir?

Cavab: Linked List (xüsusilə Doubly Linked List) daha əlverişlidir: mahnının yerini dəyişmək üçün elementləri köçürmək lazım deyil, yalnız bir neçə göstərici yenidən bağlanır. Array-də isə sıranı dəyişmək elementlərin sürüşdürülməsi (O(n)) tələb edir.

B. Proqramlaşdırma

Aşağıdakı funksiyaları dəstəkləyən Playlist sistemi hazırlayın:

Song

- Song ID
- Song Name
- Artist

Əməliyyatlar

Add Song

Delete Song

Search Song

Display Playlist

Count Songs

Məhdudiyyət

Hazır STL list istifadəsi qadağandır.

Implementasiya (Python, hazır list/deque istifadə edilmədən): SongNode və Playlist sinifləri, mənbə fayl: src/playlist.py.

C. Təcrübə və Eksperiment

Playlist sistemini aşağıdakı ölçülərdə test edin:

100 mahnı

1000 mahnı

10000 mahnı

Aşağıdakı əməliyyatların vaxtını ölçün:

Insert

Search

Delete

Nəticələri aşağıdakı formada təqdim edin

Element sayı	Insert (ms)	Search (ms)	Delete (ms)
100	0.02940	0.002170	0.002431
1000	0.3616	0.02637	0.02301
10000	4.2458	0.2377	0.2725

Qeyd: Insert N elementin əlavə edilməsinin ümumi vaxtıdır (5 təkrarın medianı). Search və Delete ən pis hal üçün (siyahının sonundakı element) bir əməliyyatın orta/median vaxtıdır.

D. Analiz və Nəticələrin Şərhi

Ən azı aşağıdakı sualları cavablandırın:

1. Element sayı artdıqca hansı əməliyyat daha çox təsirləndi?

Cavab: Bir əməliyyat üçün ölçülən Search və Delete ən çox təsirləndi: element sayı 100-dən 10000-ə (100 dəfə) artdıqda Search 0.00217 ms-dən 0.2377 ms-ə (≈110 dəfə), Delete 0.00243 ms-dən 0.2725 ms-ə (≈112 dəfə) artdı. Insert-in cədvəldəki vaxtı N əməliyyatın cəmi olduğu

üçün artıb (0.0294 → 4.2458 ms), lakin bir insert-ə düşən orta vaxt demək olar sabit qalıb (≈0.29 → 0.42 µs).

2. Təcrübə nəticələri Big-O nəzəriyyəsini təsdiqləyirmi?

Cavab: Bəli. Tail göstəricisi saxlandığı üçün Insert O(1)-dir: N dəfə insert-in cəmi vaxtı təxminən xətti artır, bir əməliyyatın vaxtı isə demək olar dəyişmir (kiçik artım yaddaş ayrılması, keş və garbage collector təsiri ilə izah olunur). Search və Delete ən pis halda (sonuncu element) bütün siyahını keçdiyi üçün O(n)-dir: element sayı hər dəfə ~10 dəfə artdıqda vaxt da təxminən 10 dəfə artır (0.0022 → 0.026 → 0.238 ms). Log-log qrafikdə Search və Delete xətləri 1-ə yaxın maillik göstərir.

3. Eyni problemi Array istifadə etməklə həll etsəydiniz nəticələr necə olardı?

Cavab: Array (Python-da list) istifadə edilsəydi, sona əlavə amortizasiya olunmuş O(1) olardı, ID üzrə axtarış yenə O(n) qalardı, lakin elementlər ardıcıl yaddaşda olduğundan keş səmərəliliyi daha yaxşı olar və sabit əmsal kiçik olardı. Sonuncu elementin silinməsi O(1), baş və ya ortadakı elementin silinməsi isə elementlərin sürüşdürülməsi səbəbindən O(n) olardı. Array-də ölçü dolduqda yenidən ayırma və köçürmə (resize) lazım ola bilər.

4. Linked List-in əsas üstünlüyü nədir?

Cavab: Ölçünün dinamik olması və elementlərin sürüşdürülməsi/köçürülməsi olmadan əlavə və silmənin yalnız göstəricilərin dəyişdirilməsi ilə (mövqe məlum olduqda O(1)) həyata keçirilməsidir. Yaddaş əvvəlcədən ayrılmır, ehtiyac olduqca node-lar yaradılır.

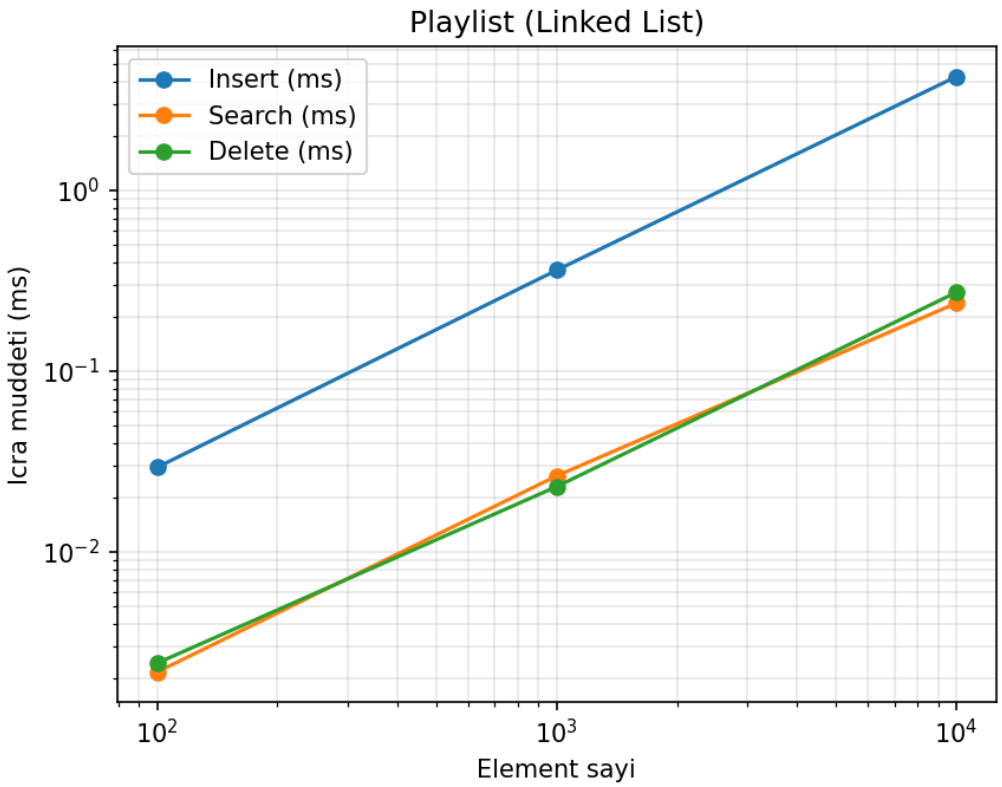
5. Playlist tətbiqində Doubly Linked List istifadə etmək daha məntiqli olardımı?

Cavab: Bəli, əksər hallarda daha məntiqli olardı. prev göstəricisi "Previous Song" düyməsini, tərs istiqamətdə gəzməyi və verilmiş node-un əvvəlki node-u axtarmadan O(1)-də silinməsini mümkün edir. Çatışmazlığı hər node üçün əlavə göstərici yaddaşı və daha mürəkkəb kod olmasıdır. Bu laboratoriyada yalnız irəli oxuma (playNextSong) tələb olunduğu üçün Singly Linked List kifayət edir.

Qrafik

Qrafikdə göstərilməlidir:

- X oxu → Element sayı
- Y oxu → İcra müddəti



Şəkil 1. Playlist (Linked List): element sayından asılı olaraq Insert, Search və Delete vaxtı (log-log miqyas).

Tapşırıq 2

Browser History Manager (Stack)

Real Həyat Ssenarisi

Veb brauzerdə istifadəçi müxtəlif səhifələrə keçid edir. "Back" düyməsi basıldıqda əvvəlki səhifələrə qayıdır.

Bu mexanizm Stack prinsipi ilə işləyir.

A. Araşdırma

1. Stack strukturunda LIFO prinsipi nədir?

Cavab: LIFO (Last In – First Out) “son daxil olan ilk çıxır” deməkdir. Stack-də elementlər yalnız bir ucdan, yəni top-dan əlavə (push) və çıxarılır (pop); ən son əlavə olunan element ilk götürülür.

2. Browser History sistemi nə üçün Stack istifadə edir?

Cavab: İstifadəçi səhifələri ardıcıllıqla ziyarət edir və “Back” düyməsi ən son ziyarət edilmiş (cari) səhifədən əvvəlki səhifəyə qaytarır. Yəni ən son əlavə olunan səhifə ilk çıxarılmalıdır; bu LIFO davranışıdır. Hər yeni səhifə push, “Back” isə pop əməliyyatı ilə O(1)-də həyata keçirilir.

3. Undo əməliyyatı necə Stack vasitəsilə həyata keçirilir?

Cavab: Hər yerinə yetirilən əməliyyat (məsələn, Write Paragraph, Insert Image, Change Font) ActionNode kimi stack-in üstünə push edilir. Undo seçildikdə top-dakı ən son əməliyyat pop edilərək geri qaytarılır və onun altındakı əməliyyat yeni top (cari vəziyyət) olur. Beləliklə əməliyyatlar həmişə edilmə sırasının əksinə geri qaytarılır.

4. Queue istifadə olunsaydı hansı problemlər yaranardı?

Cavab: Queue FIFO prinsipi ilə işlədiyi üçün Back/Undo ən yeni əməliyyatı deyil, ən köhnəsini (ilk ziyarət edilən səhifəni) qaytarardı. Bu, istifadəçinin gözlədiyi davranışı pozar, əməliyyatların geri qaytarılma sırası səhv olar və sənədin/sessiyanın vəziyyəti ardıcıl olmazdı.

5. Linked List və Array əsaslı Stack implementasiyalarını müqayisə edin.

Cavab: Linked List əsaslı Stack: ölçü dinamikdir və overflow yoxdur, push/pop həmişə O(1)-dir (head-də), amma hər node üçün əlavə göstərici yaddaşı və node yaratma xərci var, yaddaşda səpələnmə keşə mənfi təsir edir. Array əsaslı Stack: elementlər ardıcıl yerləşdiyi üçün yaddaşa qənaət və keş üstünlüyü var, push/pop sürətlidir; lakin ölçü sabitdirsə overflow ola bilər, dinamik array olduqda isə dolduqda nadir hallarda O(n) resize baş verir (amortizasiya olunmuş O(1)).

B. Proqramlaşdırma

Aşağıdakı menyunu hazırlayın:

Visit Page

Back

Current Page

Display History

Exit

Məlumat
Page ID

URL

Visit Time

Məhdudiyyət

Hazır STL stack istifadəsi qadağandır.

Implementasiya (Python): variant üzrə Undo History – ActionNode (Action ID, Action Name, Time, next) və UndoStack sinifləri, mənbə fayl: src/undo_stack.py.

C. Təcrübə və Eksperiment

Aşağıdakı həcmdə əməliyyatlar yaradın:

100 Push/Pop

1000 Push/Pop

10000 Push/Pop

Ölçün:

Push müddəti

Pop müddəti

Cədvəl

Əməliyyat sayı	Push (ms)	Pop (ms)
100	0.2458	0.01300
1000	2.5130	0.1358
10000	26.3140	1.4674

Qeyd: Push və Pop N əməliyyatın ümumi vaxtıdır (5 təkrarın medianı); hər Push zamanı ActionNode yaradılır və vaxt damğası (datetime.now) hesablanır.

D. Analiz və Nəticələrin Şərhi

1. Push və Pop əməliyyatları arasında fərq müşahidə olundumu?

Cavab: Bəli, Pop Push-dan xeyli sürətlidir: 10000 əməliyyatda Push 26.31 ms, Pop 1.47 ms (≈18 dəfə fərq). Səbəb: pushAction yeni ActionNode obyekti yaradır və hər node üçün datetime.now().strftime() ilə vaxt damğası hesablayır, undoAction isə yalnız top göstəricisini növbəti node-a keçirir və ölçünü azaldır.

2. Əldə olunan nəticələr O(1) mürəkkəbliyini dəstəkləyirmi?

Cavab: Bəli. Əməliyyat sayı 10 dəfə artdıqda ümumi vaxt da təxminən 10 dəfə artır (Push: 0.246 → 2.513 → 26.314 ms; Pop: 0.013 → 0.136 → 1.467 ms), yəni bir əməliyyata düşən vaxt demək olar sabitdir (Push ≈2.5 µs, Pop ≈0.13–0.15 µs). Bu, push və pop-un stack-in ölçüsündən asılı olmayaraq O(1) olduğunu təsdiqləyir.

3. Array əsaslı Stack ilə müqayisədə hansı fərqlər yarana bilər?

Cavab: Array əsaslı Stack-də node obyekti yaradılmadığı və elementlər ardıcıl yaddaşda saxlandığı üçün push/pop adətən daha kiçik sabit əmsala və daha yaxşı keş davranışına malik olur; Python-da list.append/pop amortizasiya olunmuş O(1)-dir. Lakin array dolduqda resize (bütün elementlərin köçürülməsi) baş verə bilər, sabit ölçülü array-də isə overflow riski var. Linked List versiyasında bu problemlər yoxdur, amma hər push-da yeni node yaradılır.

4. Brauzerlərdə tarixçənin saxlanması üçün niyə Stack istifadə olunur?

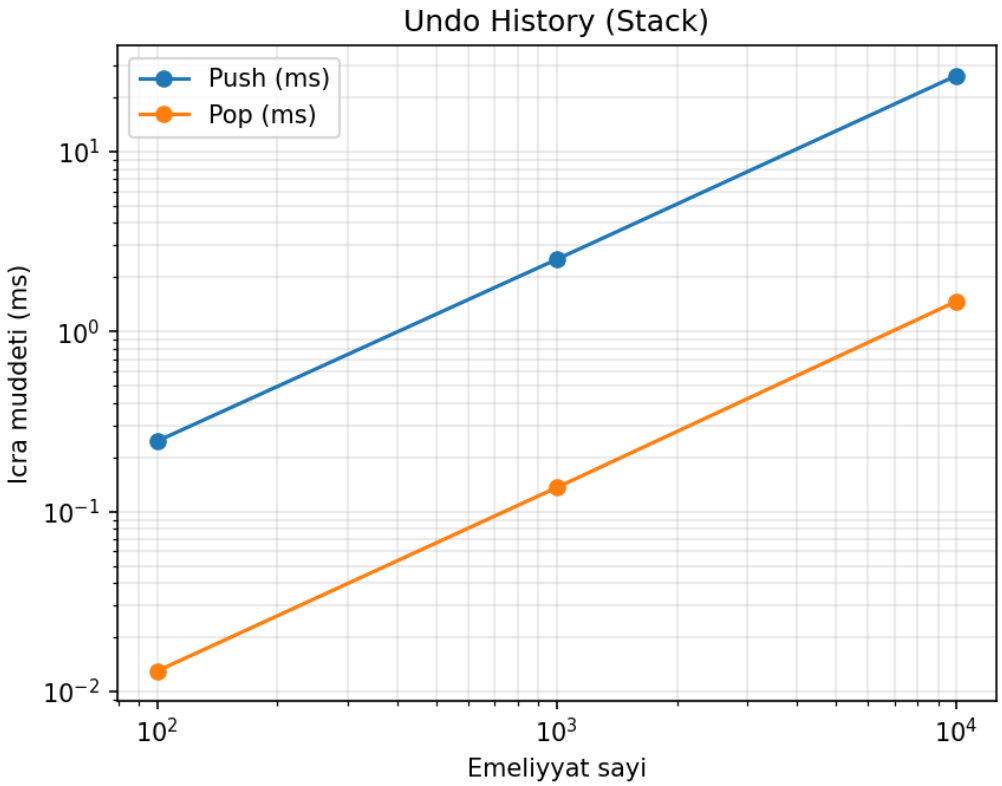
Cavab: Çünki istifadəçi həmişə ən son baxdığı səhifəyə ilk geri qayıdır; Stack ən son əlavə olunan elementə O(1)-də giriş verir və Back əməliyyatını təbii şəkildə (pop) ifadə edir. Eyni mexanizm Undo/Redo üçün də istifadə olunur.

5. Stack-in real sistemlərdə başqa hansı tətbiqləri mövcuddur?

Cavab: Funksiya çağırışlarının idarəsi (call stack, rekursiya), mətn redaktorlarında Undo/Redo, ifadələrin qiymətləndirilməsi və mötərizələrin yoxlanması, infix-dən postfix-ə çevirmə, DFS (dərinlik üzrə axtarış), geri izləmə (backtracking) alqoritmləri (labirint, sudoku) və kompilyatorların sintaksis analizi.

Qrafik

Push və Pop əməliyyatlarının performans müqayisəsini təqdim edin.



Şəkil 2. Undo History (Stack): Push və Pop əməliyyatlarının performans müqayisəsi (log-log miqyas).

Tapşırıq 3

Airport Check-in Queue System (Queue)

Real Həyat Ssenarisi

Hava limanında sərnişinlər qeydiyyatdan keçmək üçün növbəyə dayanırlar.

Birinci gələn sərnişin birinci xidmət alır (FIFO).

A. Araşdırma

1. Queue strukturunun FIFO prinsipi nədir?

Cavab: FIFO (First In – First Out) “ilk daxil olan ilk xidmət alır” deməkdir. Elementlər rear (arxa) ucundan əlavə edilir (enqueue) və front (ön) ucundan çıxarılır (dequeue).

2. Queue və Stack arasında əsas fərqlər hansılardır?

Cavab: Stack LIFO prinsipi ilə işləyir və yalnız bir ucdan (top) əməliyyat aparılır (push/pop); Queue FIFO prinsipi ilə işləyir və iki uc istifadə olunur: rear-də enqueue, front-da dequeue. Stack Undo, rekursiya, geri izləmə üçün; Queue isə növbə, planlaşdırma (scheduling), bufer və çap növbəsi kimi sistemlər üçün uyğundur.

3. Hava limanında növbə idarəetməsi üçün Queue niyə uyğundur?

Cavab: Sərnişinlər gəliş sırası ilə xidmət almalıdır: birinci gələn birinci qeydiyyatdan keçir. Queue bu ədalətli sıranı təbii şəkildə təmin edir, yeni sərnişin rear-ə qoşulur, xidmət isə front-dan başlayır; hər iki əməliyyat O(1)-dir.

4. Queue istifadə olunmasa hansı problemlər yarana bilər?

Cavab: Növbə strukturu olmasa sərnişinlərin xidmət sırası pozular: məsələn, Stack olsaydı son gələn birinci xidmət alar, əvvəl gələnlər isə çox gözləməli (starvation) olardı. Bu, ədalətsizliyə, uzun gözləmə vaxtına və sistemdə qarışıqlığa gətirib çıxarardı.

5. Circular Queue nə kimi üstünlüklər təqdim edir?

Cavab: Circular Queue array əsaslıdır və rear/front indeksləri sona çatdıqda modulo əməliyyatı ilə massivin əvvəlinə qayıdır. Beləliklə dequeue-dan sonra boşalan yerlər yenidən istifadə olunur, elementlərin sürüşdürülməsi lazım olmur, yaddaş sabit qalır və enqueue/dequeue O(1) işləyir. Adi array queue-dakı “yer var, amma növbə dolu görünür” problemi aradan qalxır.

B. Proqramlaşdırma

Aşağıdakı sistemi yaradın:

Passenger
Passenger ID

Name

Flight Number

Əməliyyatlar
Add Passenger

Check-In Passenger

Peek Next Passenger

Display Queue

Count Passengers

Implementasiya (Python): variant üzrə University Printing Service – PrintJobNode və PrintQueue sinifləri, mənbə fayl: src/print_queue.py.

C. Təcrübə və Eksperiment

Süni olaraq aşağıdakı ölçülərdə növbələr yaradın:

100 sərnişin

1000 sərnişin

10000 sərnişin

Ölçün:

Enqueue

Dequeue

Peek

Cədvəl

Element sayı	Enqueue (ms)	Dequeue (ms)	Peek (ms)
100	0.02160	0.01240	0.005100
1000	0.2638	0.1212	0.03810
10000	2.7776	1.3248	0.3876

Qeyd: Enqueue, Dequeue və Peek N əməliyyatın ümumi vaxtıdır (5 təkrarın medianı); Peek üçün N dəfə peek() çağırılmışdır. Tapşırıq variantına uyğun olaraq növbə PrintQueue (çap işləri) ilə həyata keçirilmişdir.

D. Analiz və Nəticələrin Şərhi

1. FIFO prinsipi praktiki nəticələrdə necə müşahidə olunur?

Cavab: Növbəyə ilk əlavə olunan iş həmişə ilk çıxarılır: nümunə ssenaridə Ali, Leyla, Nigar sırası ilə əlavə edildikdə “Print Next Job” ilk olaraq Ali-ni çap edir və növbə FRONT → Leyla, Nigar → REAR şəklinə düşür. peek() isə həmişə front-dakı elementi göstərir və onu silmir; dequeue yalnız front göstəricisini dəyişir, rear isə yalnız növbə boşaldıqda sıfırlanır.

2. Queue əməliyyatlarının mürəkkəbliyi nəzəri nəticələrə uyğundurmu?

Cavab: Bəli. Enqueue, dequeue və peek O(1)-dir (front və rear göstəriciləri saxlanır). Bir əməliyyata düşən vaxt element sayından asılı olmayaraq təxminən sabitdir: Enqueue ≈0.22–

0.28 µs, Dequeue ≈0.12–0.13 µs, Peek ≈0.04–0.05 µs; N əməliyyatın ümumi vaxtı isə N ilə təxminən xətti artır (Enqueue: 0.0216 → 0.2638 → 2.7776 ms).

3. Queue əvəzinə Stack istifadə olunsa nə baş verərdi?

Cavab: Stack LIFO olduğu üçün ən son gələn sərnişin (çap işi) birinci xidmət alardı, ilk gələnlər isə növbənin dibində qalardı. Bu, FIFO ədaləti tələb edən sistemdə yanlış sıra və uzun gözləmə (starvation) yaradardı.

4. Növbə ölçüsü artdıqca hansı dəyişikliklər müşahidə olundu?

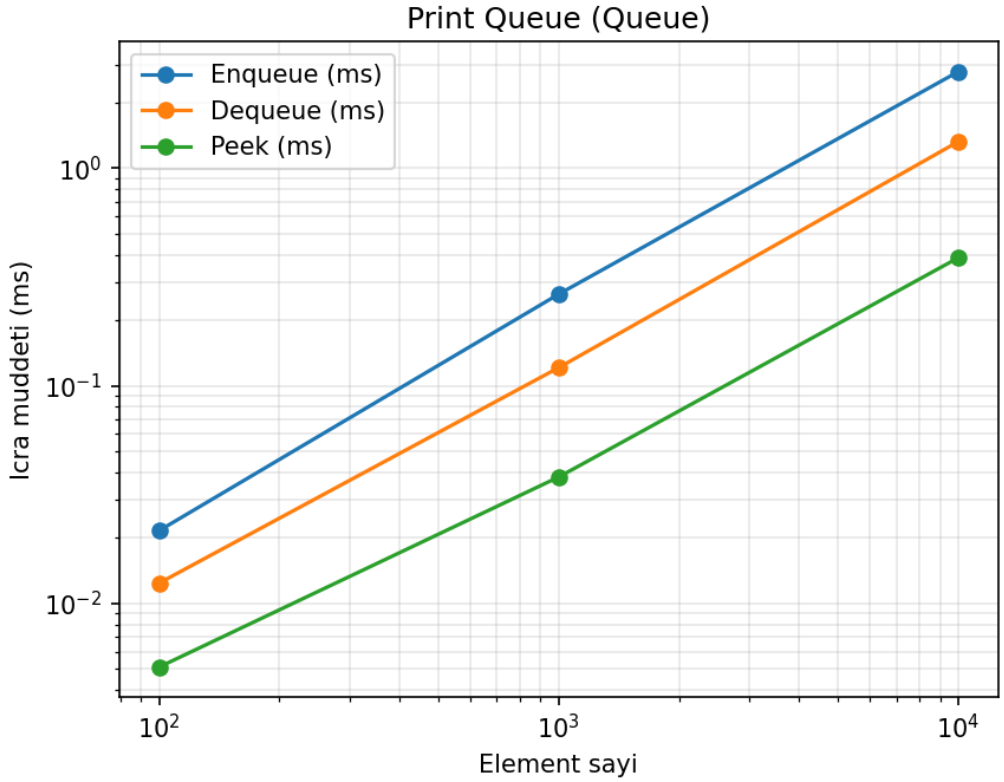
Cavab: Ölçü artdıqca əməliyyatların ümumi vaxtı təxminən xətti artdı (10 dəfə çox element → təxminən 10 dəfə çox vaxt), bir əməliyyatın orta vaxtı isə demək olar dəyişmədi. Enqueue dequeue-dan təxminən 2 dəfə yavaş oldu, çünki yeni node obyekti yaradılır; peek ən sürətli əməliyyatdır, çünki yalnız front-a istinad qaytarır. Kiçik sapmalar yaddaş ayrılması, keş və garbage collector təsiri ilə bağlıdır.

5. Circular Queue istifadə etməyin üstünlükləri nələrdir?

Cavab: Yaddaşın sabit ölçüdə əvvəlcədən ayrılması və boşalan yerlərin təkrar istifadəsi, dequeue zamanı elementlərin sürüşdürülməsinin olmaması, enqueue/dequeue-nun O(1) qalması, keş baxımından yaxşı davranış və bufer (məsələn, klaviatura, şəbəkə, audio bufer) tətbiqlərinə uyğunluq. Linked List əsaslı Queue-da ölçü dinamik olduğu üçün bu problem yaranmır, lakin node yaratma xərci var.

Qrafik

Element sayı ilə əməliyyat vaxtları arasındakı əlaqəni qrafiklə göstərin



Şəkil 3. Print Queue (Queue): Enqueue, Dequeue və Peek vaxtlarının element sayından asılılığı (log-log miqyas).

Sistem Məlumatları (Məcburidir)

Bütün analizlərdə aşağıdakı məlumatlar göstərilməlidir:

Computer Model: Acer Nitro Lite 16

CPU: Intel Core i5-13420H

RAM: 16 GB

GPU: NVIDIA GeForce RTX 3050

Operating System: Windows 11 Pro

Compiler: Python (CPython interpretator; C++ kompilyatoru istifadə olunmayıb)

Compiler Version: Python 3.12

IDE: Visual Studio Code

Screen Resolution: 1920x1200

Storage Type (SSD/HDD): NVMe SSD

Power Mode: Balanced

Other Factors: time.perf_counter() ilə ölçmə, hər ölçmə 5 dəfə təkrarlanıb və median götürülüb; Python-un garbage collector-u; arxa planda işləyən proqramlar.

Nümunə
Computer Model: Lenovo ThinkPad E14

CPU: Intel Core i5-1235U

RAM: 16 GB

GPU: Intel Iris Xe

OS: Windows 11 Pro

Compiler: GCC

Version: 13.2

Resolution: 1920x1080

Storage: NVMe SSD

Analiz zamanı bu parametrlərin nəticələrə necə təsir edə biləcəyi izah edilməlidir.

Parametrlərin nəticələrə təsiri: CPU-nun taktı, nüvə sayı və turbo/istilik məhdudiyyəti (thermal throttling) vaxtları birbaşa dəyişir – eyni kod daha sürətli prosessorda daha qısa vaxt göstərir. RAM və CPU keşi node-ların yerləşməsinə təsir edir: Linked List/Stack/Queue node-ları yaddaşda səpələndiyi üçün keş buraxılışları (cache miss) artır və element sayı böyüdükcə Search/Delete üçün sabit əmsal dəyişə bilər. Power Mode (Battery saver / Performance) prosessor tezliyini dəyişərək nəticələri dəyişir. Əməliyyat sistemi planlaşdırıcısı və arxa plan prosesləri ölçmədə təsadüfi sapmalar yaradır, ona görə hər ölçmə 5 dəfə təkrarlanıb və median götürülüb. Python interpretatoru (CPython) və garbage collector node obyektlərinin yaradılma xərcini artırır; C++ ilə eyni testlər xeyli sürətli olardı. Storage tipi (SSD/HDD), GPU və ekran ölçüsü yalnız yaddaşda aparılan bu əməliyyatlara praktiki təsir göstərmir.

Hesabat Strukturı

Tələbə PDF formatında hesabat təqdim etməlidir.

Hesabat bölmələri

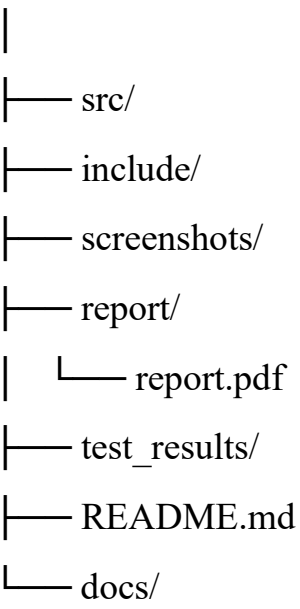
1. Giriş
2. Tapşırıq 1
3. Tapşırıq 2
4. Tapşırıq 3
5. Test nəticələri
6. Cədvəllər
7. Qrafiklər

8. Analiz
9. Nəticə
10. İstinadlar

GitHub Tələbləri

Repository aşağıdakı struktura malik olmalıdır:

Lab02/



Minimum tələblər

- Minimum 5 meaningful commit
- README faylı
- Tam kompilyasiya olunan kod
- Screenshotlar
- Hesabat
- Test nəticələri

Süni İntellektdən İstifadə Siyasəti

Bu laboratoriya işində AI alətlərindən (ChatGPT, Copilot, Gemini və s.) istifadə yalnız aşağıdakı məqsədlərlə icazəlidir:

- Sintaksis izahı
- Compiler xətalarının izahı
- C++ və Python dil konstruksiyalarının öyrənilməsi

AI vasitəsilə yaradılmış kodun birbaşa təqdim edilməsi qadağandır.

Tələbə istifadə etdiyi bütün AI alətlərini hesabatın sonunda ayrıca qeyd etməlidir:

AI Usage Declaration

Tool:

Purpose:

Used Sections:

AI istifadəsinin bəyan edilməməsi akademik pozuntu hesab ediləcək.

Akademik Dürüstlük Qaydaları

Qadağandır:

- Başqa tələbənin kodunu köçürmək
- GitHub repozitoriyalarından hazır həll təqdim etmək
- Süni intellekt vasitəsilə yaradılmış kodu müəllif kimi təqdim etmək
- Hesabatın plagiat formada hazırlanması

Akademik pozuntu aşkar olunarsa iş qiymətləndirilmədən geri qaytarıla bilər.

Qiymətləndirmə Meyarları (100 bal)

Meyar	Bal
Araşdırma hissəsi	15
Kod keyfiyyəti və OOP yanaşması	15
Düzgün implementasiya	20
Eksperiment və testlər	15
Analiz və nəticələrin şərhi	15
Cədvəllər və qrafiklər	10
Hesabatın keyfiyyəti	5
GitHub fəaliyyəti və commitlər	5
Cəmi	**100**

Təhvil Verilən Fayllar

- GitHub Repository Linki
- PDF Hesabat (yazı qaydaları və nümunə nəzərə alın, BlackBoard yüklənməlidir.)
- Mənbə Kodları
- Test Nəticələri
- Qrafiklər və Screenshotlar

Qeyd: Bu laboratoriya işi yalnız proqramlaşdırma bacarığını deyil, həm də tələbənin verilənlər strukturlarını araşdırmaq, eksperimental şəkildə qiymətləndirmək və

nəticələri elmi əsaslandırmaq qabiliyyətini ölçür. Bu yanaşma beynəlxalq universitetlərdə tətbiq edilən araşdırma və analiz yönümlü Data Structures laboratoriyalarına uyğun hazırlanmışdır.

Hər variantda proqramlaşdırma hissələrinin izahı.

Tapşırıq 1: Spotify Playlist Manager (Linked List)

B. Proqramlaşdırma

Bu tapşırıqda siz musiqi playlistini idarə edən sadə bir proqram hazırlayacaqsınız.

Linked List strukturu vasitəsilə mahnılar bir-birinə bağlanmalıdır. Hər mahnı ayrıca bir node (qovşaq) kimi saxlanmalıdır.

Yaradılacaq siniflər

SongNode

Aşağıdakı məlumatları saxlamalıdır:

Song ID

Song Name

Artist

Duration

Pointer to Next Song

Playlist

Playlist sinfi aşağıdakı funksiyaları təmin etməlidir:

addSong()

deleteSong()

searchSong()

displayPlaylist()

countSongs()

playNextSong()

Proqramın işləmə ssenarisi

İstifadəçi menyudan seçim etməlidir:

1. Add Song

2. Delete Song

3. Search Song

4. Display Playlist

5. Play Next Song

6. Count Songs

7. Exit

Nümunə
Add Song

ID: 1

Name:

Artist: Imagine Dragons

Song added successfully.

Tələbə öyrənəcək

- Linked List node-larının yaradılması
- Pointer istifadəsi
- Traversal
- İstənilən elementin əlavə edilməsi və silinməsi
- Dinamik yaddaş idarəetməsi

Tapşırıq 2: Browser Undo History (Stack)

B. Proqramlaşdırma

Bu tapşırıqda mətn redaktoru və ya brauzerdəki Undo funksiyasını modelləşdirəcəksiniz.

Stack prinsipi:

Last In ->First Out (LIFO)

ən son edilən əməliyyat ilk geri qaytarılır.

Yaradılacaq siniflər

ActionNode

Saxlanılacaq məlumatlar:

Action ID

Action Name

Time

Pointer to Next Action

UndoStack

Aşağıdakı funksiyaları yaratmalısınız:

pushAction()

undoAction()

currentAction()

displayHistory()

countActions()

Proqramın işləmə ssenarisi

İstifadəçi aşağıdakı əməliyyatları edə bilməlidir:

Perform Action

Undo Action

View Current Action

Display Full History

Exit

Nümunə

Perform Action: Write Paragraph

Perform Action: Insert Image

Perform Action: Change Font

Stack vəziyyəti:

TOP

Change Font

Insert Image

Write Paragraph

İstifadəçi Undo etdikdə:

Undo: Change Font

və həmin element Stack-dən çıxarılmalıdır.

Tələbə öyrənəcək

- Stack ADT
- Push və Pop əməliyyatları
- Top anlayışı
- LIFO prinsipi
- Undo sistemlərinin işləmə prinsipi

Tapşırıq 3: University Printing Service (Queue)

B. Proqramlaşdırma

Bu tapşırıqda universitet laboratoriyasındakı printer növbəsini modelləşdirəcəksiniz.

Queue prinsipi:

First In ->First Out (FIFO)

ilk daxil olan çap işi ilk emal edilir.

Yaradılacaq siniflər

PrintJobNode

Saxlanılacaq məlumatlar:

Job ID

Student Name

Number of Pages

Pointer to Next Job

PrintQueue

Aşağıdakı funksiyalar dəstəklənməlidir:

enqueue()

dequeue()

peek()

displayQueue()

countJobs()

Proqramın işləmə ssenarisi

İstifadəçi aşağıdakı seçimləri etməlidir:

1. Add Print Job

2. Print Next Job

3. View Next Job

4. Display Queue

5. Count Jobs

6. Exit

Nümunə

Çap növbəsinə aşağıdakı işlər əlavə edilir:

Ali - 5 pages

Leyla - 10 pages

Nigar - 3 pages

Queue görünüşü:

FRONT

Ali

Leyla

Nigar

REAR

İstifadəçi "Print Next Job" seçdikdə:

Ali printed successfully.

Queue belə dəyişir:

FRONT

Leyla

Nigar

REAR

Tələbə öyrənəcək

- Queue ADT
- Front və Rear göstəriciləri

- Enqueue və Dequeue əməliyyatları
- FIFO prinsipi
- Növbə əsaslı sistemlərin işləmə məntiqi

Əlavə qeyd (Bütün tapşırıqlar üçün)

C++ istifadə edən tələbələr:

std::list

std::stack

std::queue

std::deque

istifadə edə bilməzlər.

Python istifadə edən tələbələr:

collections.deque

queue.Queue

queue.LifoQueue

queue.PriorityQueue

istifadə edə bilməzlər.

Tələbələr bütün strukturları:

- öz Node sinifləri ilə,
- pointer/reference əlaqələri ilə,
- sıfırdan implementasiya etməlidirlər.

Bu yanaşma Weiss və Goodrich kitablarında vurğulanan əsas məqsədə, yəni tələbənin verilənlər strukturunun daxilində nə baş verdiyini başa düşməsinə xidmət edir.

AI Usage Declaration

Tool: Claude AI (Anthropic)

Purpose: Araşdırma və analiz suallarının cavablarının hazırlanması, nəticələrin cədvəl və qrafiklərlə təqdim edilməsi, hesabatın formatlanması.

Used Sections: Tapşırıq 1–3 üzrə A (Araşdırma), C (cədvəl və qrafiklər) və D (Analiz) bölmələri. Proqram